

AgriConnect — Software Requirements Specification

Module: Admin Role

Document version: 1.0 **Prepared for:** Development Team **Status:** Draft for review

Companion documents: [AgriConnect_Farmer_Role_SRS.md](#),

[AgriConnect_Buyer_Role_SRS.md](#) (shares entities: Product, Booking, Report)

1. Purpose

This document specifies the functional and non-functional requirements for the **Admin role**, which operates and protects the platform without participating in the farmer-buyer transaction itself. It follows the same structure as the Farmer and Buyer SRS documents so all three can be handed to the development team as one coherent set.

2. Scope

Covers Admin account provisioning, authentication, dashboard, user management, listing moderation, booking/payment oversight, report handling, category and settings management, notifications, analytics, and audit logging. It does not cover the Farmer or Buyer end-user experience except where Admin actions directly affect them (e.g., a suspension notification).

3. Definitions and Glossary

Same base glossary as the Farmer SRS §3, plus:

Term	Meaning
Super Admin	Highest-privilege internal role; can manage other Admins and platform settings
Moderator	Restricted Admin role limited to product/report moderation
Audit Log	Immutable record of privileged actions taken by Admin users

4. Actors

Actor	Description
Admin	Primary actor for this document (includes Super Admin, Admin, Moderator, Support Admin sub-types)
Farmer	Secondary actor — subject of moderation actions
Buyer	Secondary actor — subject of moderation actions
System	Background actor — audit logging, scheduled jobs, notification dispatch

5. Admin Role Overview

High-level flow:

Admin Login → Dashboard → Manage Farmers & Buyers → Monitor Listings → Monitor Bookings & Payments → Handle Reports → Manage Categories/Settings → View Analytics → Review Audit Logs → Logout

The Admin explicitly does **not**: negotiate prices, arrange logistics, process the final agricultural product payment, guarantee product quality, or act as a broker. This mirrors the "0% mediator" boundary defined in the Farmer and Buyer SRS documents — the Admin protects the *platform*, not the *trade*.

6. Data Model

Reuses [Product](#), [Booking](#), and [Report](#) entities from the Farmer SRS (§6.2–6.4) and [Farmer/Buyer](#) entities from their respective SRS documents. New entities introduced here:

6.1 Admin

Field	Type	Required	Notes
admin_id	UUID/PK	Yes	
full_name	string	Yes	
email	string	Yes	Unique, used as login identifier

mobile_number	string	Recommended	Used for 2FA/recovery
password_hash	string	Yes	
admin_role	enum	Yes	SUPER_ADMIN, ADMIN, MODERATOR, SUPPORT_ADMIN
permissions	array<string>	Conditional	Fine-grained permission list if role-based access control goes beyond the 4 fixed tiers (see Open Question 24.10)
account_status	enum	Yes	ACTIVE, SUSPENDED, DISABLED
two_factor_enabled	boolean	Yes	Default true for SUPER_ADMIN, recommended for all
created_by	FK (admin_id)	Yes	Every Admin account must trace back to who created it — never self-registered
created_at / updated_at	datetime	Yes	
last_login_at	datetime	No	

Critical constraint: there is no public registration endpoint for this entity. Accounts are created only via an authenticated Super Admin action or a secured backend script/seed process — this should be enforced by *not exposing* a create-Admin route publicly, not merely by hiding it in the UI.

6.2 AuditLog

Field	Type	Required	Notes
log_id	UUID/PK	Yes	
admin_id	FK	Yes	
action_type	enum	Yes	e.g., SUSPEND_FARMER, REMOVE_PRODUCT, RESOLVE_REPORT, UPDATE_SETTING
target_type	enum	Yes	FARMER, BUYER, PRODUCT, BOOKING, REPORT, CATEGORY, SETTING, ADMIN
target_id	UUID	Yes	
previous_state	JSON	No	Snapshot before the change, where applicable

new_state	JSON	No	Snapshot after the change
reason	text	Conditional	Required for suspend/block/remove actions (FR-A-008)
ip_address	string	No	Where legally/operationally appropriate — see Open Question 24.7
created_at	datetime	Yes	

Constraint: this table must be append-only. No UPDATE or DELETE operations should be permitted at the application layer, and ideally the database role used by the app has no delete/update grant on this table at all.

6.3 Report — Extended (base schema in Farmer SRS §6.4)

The Admin-facing view needs the report target to be polymorphic, since both Farmer SRS and Buyer SRS describe reports being filed against a booking, but the Admin spec above also implies reports against a listing directly. Recommend generalizing:

Field	Type	Required	Notes
target_type	enum	Yes	FARMER, BUYER, PRODUCT (extends the booking-only model in Farmer SRS §6.4 — see Open Question 24.8)
target_id	UUID	Yes	References the relevant Farmer, Buyer, or Product record
booking_id	FK	No	Now optional — a report may reference a listing directly without a completed booking

6.4 Category

Field	Type	Required	Notes
category_id	UUID/PK	Yes	
name	string	Yes	Unique
icon_url	string	No	
is_active	boolean	Yes	Default true; deactivation preferred over deletion (§13)
created_at / updated_at	datetime	Yes	

6.5 PlatformSetting

Field	Type	Required	Notes
setting_key	string/PK	Yes	e.g., <code>booking_fee_amount</code> , <code>listing_expiry_days</code> , <code>max_product_images</code>
setting_value	string/JSON	Yes	
updated_by	FK (admin_id)	Yes	
updated_at	datetime	Yes	

Business rule: changing `booking_fee_amount` must only affect *new* bookings from that point forward. Existing `Booking` records already store `booking_fee_amount` at the row level (Farmer SRS §6.3), so historical amounts are naturally preserved — no separate versioning table is needed as long as that field is never backfilled or recalculated retroactively.

6.6 Notification (Announcement)

Field	Type	Required	Notes
notification_id	UUID/PK	Yes	
sent_by	FK (admin_id)	Yes	
target_audience	enum	Yes	ALL_USERS, ALL_FARMERS, ALL_BUYERS, SPECIFIC_USER
target_user_id	FK	Conditional	Required if target_audience = SPECIFIC_USER
message	text	Yes	
sent_at	datetime	Yes	

7. Admin Account Creation Workflow

7.1 Fields (see §6.1 for full schema)

Full Name, Email (unique), Mobile (recommended), Password, Admin Role, Account Status.

7.2 Functional Requirements

- The system shall not expose any public-facing Admin registration endpoint.
 - The system shall only allow Admin account creation by an authenticated Super Admin or via a secured backend/seed process.
 - The system shall require unique email addresses across all Admin accounts.
 - The system shall hash Admin passwords with the same standard as Farmer/Buyer (bcrypt/argon2).
 - The system shall log Admin account creation in the audit trail (§6.2), including which Super Admin created it.
-

8. Admin Login Workflow

8.1 Flow

Open Admin Portal → Enter Email/Mobile + Password → Authentication → 2FA (recommended, required for Super Admin) → Admin Dashboard

8.2 Functional Requirements

- The system shall serve the Admin Portal on a separate route/subdomain from the Farmer/Buyer app, ideally with additional network-level protections (IP allowlisting or VPN for production, at minimum for Super Admin).
 - The system shall rate-limit login attempts (recommend: lockout after 5 failed attempts for 15 minutes, consistent with Farmer/Buyer login rules).
 - The system shall require 2FA for all Super Admin logins and should support it as opt-in-but-encouraged for other Admin tiers.
 - The system shall auto-expire idle Admin sessions faster than end-user sessions (e.g., 30 minutes idle timeout vs. 7-day token for Farmer/Buyer) given the sensitivity of the access.
 - The system shall log every login attempt (success and failure) with timestamp and IP for security review.
-

9. Admin Dashboard Workflow

9.1 Statistics

Total/Active/Suspended Farmers; Total/Active/Suspended Buyers;
Total/Active/Sold/Removed Products; Total Successful Bookings; Pending/Successful/Failed Booking Payments; Total/Pending Reports.

9.2 Recent Activity Feed

Recent registrations (farmer/buyer), recent listings, recent successful bookings, recent reports, recent Admin actions.

9.3 Navigation

Dashboard, Farmers, Buyers, Products, Bookings, Booking Payments, Reports, Categories, Notifications, Analytics, Admin Management, Settings, Audit Logs.

9.4 Non-Functional Note

Dashboard statistics should be computed from a materialized/cached aggregate refreshed periodically (e.g., every few minutes) rather than live-queried on every dashboard load — this avoids expensive COUNT queries across large tables on every Admin page view, which is a common performance bottleneck as the platform scales.

10. Farmer Management Workflow

10.1 Information Displayed

Farmer ID, Name, Verified Mobile, Email, State/District/Village, Registration Date, Products Listed count, Bookings Received count, Account Status.

10.2 Account Statuses

Status	Meaning
ACTIVE	Normal platform access
SUSPENDED	Temporarily restricted, reversible
BLOCKED	Disabled for serious violations
DELETED	Removed per platform/data-retention rules

10.3 Admin Actions

View profile, view products, view bookings, view reports against the farmer, Suspend, Restore, Block.

10.4 Functional Requirements

- The system shall require a reason (free text or selected from a predefined list) for any suspend or block action, stored in the audit log (FR-A-008).

- The system shall notify the affected farmer of the status change and the stated reason, unless the reason is withheld for an active fraud investigation (see Open Question 24.9).
 - The system shall prevent a suspended/blocked farmer from publishing new listings or receiving new bookings, while existing confirmed bookings and their contact-unlock data remain intact for buyers who already booked (consistent with the "suspension mid-booking" edge case in Farmer SRS §23).
-

11. Buyer Management Workflow

Mirrors §10 structurally. Information: Buyer ID, Name, Mobile, Email, Buyer Type, State/District, Registration Date, Successful Bookings count, Reports Received count, Account Status.

Admin actions: View profile, view booking history, view reports, Suspend, Block, Restore.

Functional requirement: the system shall not permit the Admin to arbitrarily edit historical booking records tied to a buyer (FR-A-011) — this is a read-only oversight view, not an editing tool.

12. Product Listing Management Workflow

12.1 Information Displayed

Product ID, Name, Image, Category, Farmer ID/Name, Quantity, Unit, Expected Price, State/District, Upload Date, Status, Booking count, Report count.

12.2 Admin Actions

View full details, view uploading farmer, view related reports, Remove, Restore, Flag for review.

12.3 Removal Reasons

Fake product, misleading images, incorrect info, prohibited content, duplicate/spam, fraud, rule violation.

12.4 Flow

Products → Select Product → Review → Remove → Select Reason → Confirm → Status = REMOVED → Farmer Notified

12.5 Functional Requirement

- The system shall block any pending/in-progress booking payment on a product the moment it's set to **REMOVED**, and shall trigger the refund-eligibility check described in the Buyer SRS §22 edge case (payment mid-flight when listing status changes).
-

13. Product Moderation Approach

MVP decision (carried over from source spec): products publish immediately (**ACTIVE**) without pre-approval; Admin moderates reactively based on reports and monitoring. A gated **Farmer Submits → Pending Review → Admin Approves → Active** flow is documented here as a **future option**, not the MVP default, so the developer doesn't build unnecessary approval-queue infrastructure prematurely.

Functional requirement: the **Product.status** enum (Farmer SRS §6.2 / §12) already includes **DRAFT**, which is sufficient to support a future pre-approval flow without a schema change — a **PENDING_REVIEW** status can be added later if the moderation approach changes.

14. Booking Management Workflow

14.1 Information Displayed

Booking ID, Product ID/Name, Farmer ID/Name, Buyer ID/Name, Date/Time, Amount, Payment Status, Booking Status.

14.2 Admin Actions

Search by Booking ID/Product/Farmer/Buyer, filter by date/payment status, view details.

14.3 Critical Restriction

- The system shall provide **read-only** access to confirmed booking records for standard Admin roles.
 - The system shall not permit changing a successful payment to failed, reassigning farmer/buyer on a booking, altering the booking amount post-payment, or editing historical timestamps through normal Admin actions.
 - Any exceptional correction (e.g., a genuine data-entry bug) shall require Super Admin privilege and shall be recorded in the audit log with the previous and new state (§6.2).
-

15. Booking Payment Monitoring Workflow

15.1 Information Displayed

Payment ID, Booking ID, Buyer ID, Product ID, Amount, Gateway Reference, Date/Time, Status.

15.2 Statuses

Initiated, Pending, Successful, Failed, Refunded (if supported).

15.3 Functional Requirements

- The system shall not allow an Admin to manually mark a **FAILED** payment as **SUCCESSFUL** under normal circumstances — this bypasses the server-side gateway verification rule established in Buyer SRS §16.4 and would let an Admin unlock contact details for an unpaid booking.
 - If refund support is implemented, the system shall allow an authorized Admin to **initiate** a refund (which calls the gateway's refund API and updates status to **REFUNDED**), but shall not allow directly editing payment status to **REFUNDED** without a corresponding gateway transaction — status and money movement must stay in sync.
-

16. Report Management Workflow

16.1 Report Targets

Farmer, Buyer, or Product Listing (see extended schema in §6.3).

16.2 Information Displayed

Report ID, Reporter User ID/Role, Reported Target, Reason, Description, Date/Time, Evidence (if supported), Status.

16.3 Statuses

Open, Under Review, Resolved, Rejected, Closed.

16.4 Admin Actions

Review report and target's account/report history, add internal notes, remove listing, suspend/block user, reject invalid report, mark resolved.

16.5 Flow

Report Submitted → Admin Reviews Evidence & History → Action Taken → Reason Recorded → Status Updated → Relevant User Notified (if applicable)

16.6 Functional Requirement

- The system shall let the Admin view a user's **prior report history** (both reports filed by them and reports filed against them) directly from the report review screen, since repeat-offender patterns are the main signal for deciding suspend vs. warn vs. dismiss.
 - The system shall respect the report rate limits defined in Farmer SRS §19 and Buyer SRS §23.3 at intake, so the Admin queue isn't flooded by abuse of the reporting feature itself.
-

17. Product Category Management

Admin actions: Add, edit name, add icon, activate/deactivate.

Functional requirement: the system shall prevent hard deletion of a category that has associated products (historical or active) — deactivation only (`is_active = false`), consistent with §6.4. Deactivated categories should be hidden from new-listing forms but remain valid on existing historical listings for display/reporting purposes.

18. Notification and Announcement Management

Target audiences: All Users, All Farmers, All Buyers, Specific User.

Functional requirement: the system shall retain a record of all sent announcements (§6.6) for audit and to avoid accidental duplicate sends. Bulk sends (ALL_USERS on a large user base) should be queued/batched rather than sent synchronously to avoid blocking the Admin request or overwhelming the SMS/push provider's rate limits.

19. Admin Analytics Workflow

19.1 Categories

User analytics (registrations, active users, growth), Product analytics (totals, by category/region), Booking analytics (totals, success/failure, trends, most-booked categories), Platform revenue analytics (booking-fee revenue only — daily/monthly, success/failure counts, refund totals).

19.2 Functional Requirement

- The system shall clearly label revenue analytics as **AgriConnect's booking-fee revenue**, distinct from the (unknown-to-the-platform) value of underlying agricultural trades, in every UI label and exported report — this prevents internal or external stakeholders from misreading platform fee revenue as GMV.
 - Analytics queries should run against a read replica or a periodically refreshed reporting table, not the live transactional database, once data volume grows.
-

20. Admin Management and Permission Levels

Admin Type	Responsibilities
Super Admin	Full access, including Admin management and platform settings
Admin	User, product, booking, and report management
Moderator	Product and report moderation only
Support Admin	User support, limited account access

MVP recommendation (carried over): a single Admin role tier is sufficient at launch; the `admin_role` enum and `permissions` field in §6.1 are designed so tiered access can be added later without a schema migration.

Functional requirement: the system shall enforce permission checks server-side on every Admin endpoint based on `admin_role` (and `permissions` if fine-grained control is introduced) — never rely on the Admin UI simply hiding buttons for lower-tier roles, since that's not a real access control.

21. Audit Log Workflow

Logged actions (non-exhaustive): farmer/buyer suspension, account blocking, product removal/restoration, report resolution, category modification, Admin account creation, permission changes, platform setting changes.

Functional requirements:

- The system shall write an audit log entry for every action listed above, synchronously with the action itself (not best-effort/async-and-forget), so an action can't succeed without being logged.
- The system shall make the AuditLog table append-only at the database permission level, not just by application convention (§6.2).

- The system shall retain audit logs for a defined minimum period per the platform's data-retention policy (see Open Question 24.7).
-

22. Platform Settings Workflow

Configurable settings: booking fee amount, supported categories, listing expiration period, max product images, supported quantity units, notification settings, maintenance mode, Terms & Conditions version, Privacy Policy version.

Functional requirement: the system shall restrict platform-settings changes to the Super Admin tier only, and shall apply changes only to new records going forward (§6.5) — never retroactively to existing bookings/listings, except where a setting is explicitly designed to be applied retroactively (e.g., extending all active listings' expiry during a maintenance event), which must be a distinct, clearly-labeled action.

23. Admin Logout and Session Security

- The system shall terminate the session and invalidate tokens on logout.
 - The system shall auto-logout idle Admin sessions (§8.2).
 - The system shall require re-authentication (password or 2FA re-prompt) for high-sensitivity actions — recommend this specifically for: creating a new Admin account, changing platform settings, and issuing a refund.
-

24. Complete Admin Journey (End-to-End)

Admin Portal Login → Credentials + 2FA → Dashboard → Manage Farmers/Buyers → Monitor Listings → Remove Fraudulent Products → Monitor Bookings → Review Payment Statuses → Handle Reports → Suspend/Block Violators → Manage Categories/Notifications → View Analytics → Review Audit Logs → Manage Settings → Secure Logout

25. Functional Requirements Summary

ID	Requirement
FR-A-001	The system shall provide secure Admin authentication, separate from the Farmer/Buyer login flow.

- FR-A-002 The system shall prevent public users from registering as Admins.
- FR-A-003 The system shall provide an Admin Dashboard with platform statistics, refreshed from a cached/aggregated source.
- FR-A-004 The system shall allow authorized Admins to view and manage Farmer accounts.
- FR-A-005 The system shall allow authorized Admins to view and manage Buyer accounts.
- FR-A-006 The system shall allow Admins to view all product listings, active or not.
- FR-A-007 The system shall allow authorized Admins to remove or restore product listings.
- FR-A-008 The system shall require a recorded reason for product removal and user suspension/block actions.
- FR-A-009 The system shall allow Admins to view booking records with search/filter.
- FR-A-010 The system shall allow Admins to view booking-payment records and statuses.
- FR-A-011 The system shall prevent unauthorized modification of historical booking and payment records, restricting exceptions to Super Admin with audit logging.
- FR-A-012 The system shall allow Admins to review reports filed against Farmers, Buyers, or Products.
- FR-A-013 The system shall allow authorized Admins to suspend, block, or restore user accounts.
- FR-A-014 The system shall allow Admins to manage product categories, with deactivation preferred over deletion.
- FR-A-015 The system shall allow authorized Admins to send platform announcements, queued for bulk sends.
- FR-A-016 The system shall provide user, product, booking, and booking-fee analytics, clearly distinguishing platform fee revenue from trade value.
- FR-A-017 The system shall record critical Admin actions in an append-only audit log, written synchronously with the action.
- FR-A-018 The system shall support configurable platform settings, applied prospectively unless explicitly designed otherwise.

FR-A-019	The system shall enforce role-based permissions server-side for all Admin actions.
FR-A-020	The system shall not allow Admins to participate in farmer-buyer negotiation, logistics, transportation, product inspection, or final payment.
FR-A-021	The system shall require 2FA for Super Admin accounts and auto-expire idle Admin sessions faster than end-user sessions.
FR-A-022	The system shall prevent an Admin from manually marking a failed payment as successful, or a refund status without a corresponding gateway transaction.

26. Non-Functional Requirements

Category	Requirement
Security	Separate Admin portal/route; 2FA for Super Admin; IP allowlisting recommended for production; rate-limited login; server-side permission enforcement on every endpoint
Privacy	Admin access to Farmer/Buyer PII should itself be logged (who viewed what), not just who <i>changed</i> what, if the platform later needs to demonstrate data-access accountability
Performance	Dashboard and analytics views backed by cached/aggregated or read-replica data, not live queries on transactional tables (§9.4, §19.2)
Reliability	Audit logging must be synchronous and cannot be bypassed by a failed downstream step (§21)
Auditability	This entire role exists largely <i>for</i> auditability — append-only logs, mandatory reasons on destructive actions, immutable historical booking/payment records
Scalability	Bulk notification sends queued/batched (§18); reporting queries isolated from transactional load path

27. Error Handling & Edge Cases

1. **Admin removes a product with an active booking:** must not silently orphan the booking — buyer should be notified per §12.5, and any in-flight payment must be checked against the refund policy (Open Question 24.1, carried from Buyer SRS).
2. **Two Admins act on the same report simultaneously:** use optimistic locking or a "claimed by" field on the Report entity so two Admins don't issue conflicting resolutions (e.g., one dismisses while another suspends).

3. **Super Admin account is compromised:** the audit log (append-only, per §21) is the primary forensic tool — this reinforces why it must not be editable even by the compromised account itself.
 4. **Booking-fee setting changed mid-transaction:** a buyer who started checkout at the old fee should complete at the old fee, not have the amount change under them — resolve by locking the fee amount into the booking record at `PAYMENT_INITIATED`, not at final confirmation.
 5. **Admin suspends a farmer who has pending, unconfirmed bookings (payment still processing):** define whether those in-flight payments should be auto-failed/refunded or allowed to complete — recommend auto-fail/refund, consistent with the "product went unavailable mid-payment" logic in Buyer SRS §22.
 6. **Category deactivated while products still reference it:** existing listings must continue to render correctly (§17) — don't let a deactivated category break existing product detail pages.
-

28. Open Questions for Product Owner

Admin-specific additions to the running list started in the Farmer SRS (§24) and Buyer SRS (§30):

#	Question
24.7	What is the data-retention period for audit logs, and does GDPR-style or India's DPDP Act data-handling apply to storing Admin IP addresses and Farmer/Buyer PII access logs?
24.8	Should reports be filed against a specific booking only (as the Farmer/Buyer SRS documents currently define it), or also against a listing/user directly with no booking involved, as this Admin spec implies? This needs to be resolved consistently across all three documents — recommend generalizing the Report entity per §6.3 above.
24.9	When a farmer/buyer is suspended for a fraud investigation, should the reason be shown to them immediately, or withheld until the investigation concludes?
24.10	Is a fixed 4-tier Admin role model (Super Admin / Admin / Moderator / Support Admin) sufficient long-term, or will the platform need granular, per-action permissions (e.g., "can remove products but not suspend users")? This affects whether <code>permissions</code> in §6.1 is needed at MVP or can be deferred.
24.11	Who is authorized to initiate a refund once refund support exists — any Admin, or only Super Admin — and does that answer depend on the refund reason (system error vs. investigated fraud)?

29. Traceability Note

FR-A-### IDs in §25 should be cross-checked against FR-F-### (Farmer SRS) and FR-B-### (Buyer SRS) wherever they govern the same entity — particularly booking/payment status transitions, which all three roles touch but only the Buyer flow should ever set to **CONFIRMED**, and only the payment gateway/reconciliation job should set to **REFUNDED**.